



Design | Innovate | Create

**Mtel (Thailand) Co., Ltd.**

**For**

**T.C.C. Technology Co., Ltd.**

**One Bangkok Mobile Application  
Invitation Code & Upgrade Membership  
Technical Design Document  
To  
T.C.C. Technology Co., Ltd.**

Total 25 sheets (w/h cover sheet)

| <br>Design   Innovate   Create | PROJECT NAME                                                                                    | JOB NO. | DOCUMENT NO. | REV. |
|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------|--------------|------|
|                                                                                                                   | One Bangkok Mobile Application - Invitation Code & Upgrade Membership Technical Design Document |         |              | 01   |

## Content table

|                                                         |    |
|---------------------------------------------------------|----|
| 1. Document Purpose                                     | 4  |
| 2. Project Background & Overview                        | 5  |
| 3. Goals & Non-Goals                                    | 6  |
| 3.1. Goals                                              | 6  |
| 3.2. Non-Goals                                          | 6  |
| 4. System Architecture                                  | 7  |
| 4.1. Architectural Overview                             | 7  |
| 4.2. Component Architecture                             | 7  |
| 4.2.1. Controller Layer                                 | 7  |
| 4.2.2. Service Layer                                    | 7  |
| 4.2.3. Repository Layer                                 | 8  |
| 4.2.4. External Integration Layer                       | 8  |
| 4.2.5. Database Layer                                   | 8  |
| 4.2.6. Cache Layer                                      | 9  |
| 5. Functional Design                                    | 9  |
| 5.1. Invitation Code                                    | 9  |
| 5.1.1. Login with Invitation Code                       | 9  |
| 5.1.2. Disable Name Editing for Invited User            | 12 |
| 5.1.3. Generate Invitation Codes                        | 13 |
| 5.2. Upgrade Membership                                 | 13 |
| 5.2.1. Upgrade Membership with Code                     | 13 |
| 5.2.2. Generate Invitation Codes                        | 15 |
| 6. Data Flow                                            | 16 |
| 6.1. Invitation Code                                    | 16 |
| 6.1.1. Registration with invitation-code Flow           | 16 |
| 6.1.2. Table ER - Diagram                               | 17 |
| 6.1.3. Generate Invitation Code Flow                    | 18 |
| 6.2. Upgrade Membership                                 | 19 |
| 6.2.1. Upgrade Membership Flow                          | 19 |
| 6.2.2. Generate Code Upgrade Flow                       | 20 |
| 7. Example Request/Response                             | 21 |
| 7.1. Request - Invitation Code Registration             | 21 |
| 7.1.1. Response (Success - 200)                         | 22 |
| 7.1.2. Response (Error - 400 - Invalid Code Format)     | 22 |
| 7.1.3. Response (Error - 404 - Code Not Found)          | 22 |
| 7.1.4. Response (Error - 409 - Duplicate Identity)      | 22 |
| 7.1.5. Response (Error - 500 - SSO Registration Failed) | 22 |
| 7.1.6. Error Scenarios                                  | 23 |
| 7.1.7. HTTP Status Codes                                | 23 |
| 7.2. Request - Generate Invitation code                 | 23 |

|                                                 |    |
|-------------------------------------------------|----|
| 7.2.1. Response (Success - 200)                 | 24 |
| 7.2.2. Response (Error - 500)                   | 24 |
| 7.2.3. Error Scenarios                          | 24 |
| 7.3. Request - Upgrade Membership               | 24 |
| 7.3.1. Response (Success - 200)                 | 24 |
| 7.3.2. Error Scenarios                          | 25 |
| 7.4. Request - Generate Upgrade Membership code | 25 |
| 7.4.1. Response (Success)                       | 25 |
| 7.4.2. Response (Error)                         | 25 |

## One Bangkok Mobile Application - Invitation Code and Upgrade Membership

To

**T.C.C. Technology Co., Ltd.**

This document serves as the official Technical Design Document (TDD) for the implementation of the Invitation Code and Upgrade Membership features within the One Bangkok Application, leveraging the Single Sign-On (SSO) platform as the primary source of truth. The initiative formalizes the migration of code generation, validation, and user identity management from the existing BZB web-based implementation to a fully native in-app experience integrated with SSO.

The primary objective is to establish a secure, centralized, and scalable architecture for managing invitation and upgrade membership codes while improving user experience and operational efficiency. By storing and validating codes within SSO, the solution ensures consistent identity handling, accurate tier assignment, and reliable synchronization with downstream systems.

This document describes the proposed architecture, functional design, technical approach, and delivery responsibilities to align all stakeholders at T.C.C. Technology Co., Ltd. and to support the successful implementation and deployment of the solution.

### 1. Document Purpose

The purpose of this Technical Design Document (TDD) is to define the technical architecture, system interactions, data structures, and implementation approach for the Invitation Code and Upgrade Membership features in the One Bangkok Application.

This document provides detailed guidance on how these features will be designed and implemented using the Single Sign-On (SSO) platform as the primary source of truth for code generation, validation, user identity management, and tier assignment. It serves as a reference for development, quality assurance, security review, and operations teams to ensure consistent understanding and execution.

Additionally, this TDD establishes design principles, technical assumptions, dependencies, and constraints, enabling stakeholders to assess feasibility, identify risks, and align on implementation standards prior to development.

The document is intended to support successful delivery of a secure, scalable, and maintainable solution that meets business requirements and enhances overall user experience.

## 2. Project Background & Overview

Currently, the Invitation Code and Upgrade Membership functionalities are operated through a BZB web-based implementation, where code generation, validation, and tier assignment are managed outside of the One Bangkok Application and Single Sign-On (SSO) ecosystem. This architecture introduces dependency on external systems, limited visibility into user identity states, and inconsistent synchronization of membership tiers across platforms.

With the One Bangkok Application now fully integrated with the SSO platform, SSO has become the primary source of truth for user authentication, identity attributes, and persona management. To align with this architecture, the Invitation Code and Upgrade Membership features will be migrated to leverage SSO for code storage, validation, and user-tier mapping.

This initiative aims to transition both features into a native in-app experience, replacing BZB webview flows with SSO-integrated services and application user interfaces. The target solution provides centralized management of codes, consistent identity handling, and reliable tier assignment, while improving user experience, security, and operational efficiency.

At a high level, the solution consists of:

- SSO services for generating, storing, and validating Invitation Codes and Upgrade Membership Codes
- Mobile application UI flows for code entry and verification
- Backend integration between SSO and downstream systems to synchronize tier and role updates

### 3. Goals & Non-Goals

#### 3.1. Goals

The goals of this technical design are to:

- Establish Single Sign-On (SSO) as the primary source of truth for Invitation Codes and Upgrade Membership Codes, including generation, storage, validation, and status management.
- Enable secure and scalable validation of codes through SSO services.
- Support native in-app user flows for Invitation Code login and Upgrade Membership code entry without relying on external webview solutions.
- Ensure consistent assignment and update of user tiers (Influencer, Ambassador) and personas based on code configuration.
- Improve reliability of identity data and synchronization between SSO and downstream systems.
- Provide a foundation that supports future expansion of code-based campaigns or membership upgrade mechanisms.
- Maintain compatibility with existing authentication, verification (email/phone), and password setup flows.

#### 3.2. Non-Goals

The following items are explicitly out of scope for this design:

- Redesign of the overall SSO authentication architecture.
- Changes to existing business rules for tier benefits, privileges, or reward calculation.
- Migration or remediation of historical codes generated by BZB.
- Implementation of admin or back-office UI for code generation (codes will be generated via API / tools such as Postman as part of this phase).
- Modifications to existing email or SMS verification mechanisms.
- Changes to campaign notification logic outside of triggering based on successful code application.

## 4. System Architecture

### 4.1. Architectural Overview

The Invitation Code and Upgrade Membership features are implemented within the IAM service using a layered architecture pattern:

- Controller Layer – Handles HTTP request/response
- Service Layer – Contains business logic and orchestration
- Repository Layer – Handles database operations
- External Integration Layer – Handles communication with BZB
- Database Layer – Stores code and activation records
- Cache Layer – Stores user card information (BZB cache)

IAM acts as the orchestration layer and enforces validation, state management, and transaction consistency.

### 4.2. Component Architecture

#### 4.2.1. Controller Layer

- File: `src/controllers/retail_code_controller.ts`
- Responsibilities:
  - Validate required headers (`x-account-id`, `x-access-token`)
  - Validate request body (`code`)
  - Map service response to HTTP response
  - Standardize error handling
- Endpoints:
  - POST `/retail-code/upgrade-membership`
  - POST `/retail-code/generate-code/upgrade-member-codes`

#### 4.2.2. Service Layer

- File: `src/services/retail_code_service/index.ts`
- Key Method:
  - `activateUpgradeMemberCode`
  - `createMemberCodes`
- Responsibilities:
  - For Upgrade Membership:
    - Validate retail code (existence, expiry, activation status)
    - Retrieve profile by `account_id`
    - Validate `profile.sso_id`
    - Call BZB external API (`user/register_invite`)
    - Update code activation record
    - Clear related cache
    - Return result

- For Code Generation:
  - Validate tier input
  - Generate random code (A-Z, 0-9, length 10)
  - Set `expires_at` = 31 Dec 23:59:59.999 (current year)
  - Insert using `createMany`
  - Generate CSV response

#### 4.2.3. Repository Layer

- File:
  - `src/repositories/upgrade_member_code_repository.ts`,
  - `src/repositories/profile_repository.ts`
- Responsibilities:
  - Query `upgrade_member_codes`
  - Update activation fields:
    - `account_id`
    - `activated_at`
  - Retrieve profile with `sso_id`

#### 4.2.4. External Integration Layer

- File: `src/libs/bzb_auth_client.ts`
- External API: `user/register_invite`
- Responsibilities:
  - Trigger tier upgrade in BZB
  - Return response containing `data.UserId`
  - Throw error if integration fails

#### 4.2.5. Database Layer

Table: `upgrade_member_codes`

| Field        | Type                | Description             |
|--------------|---------------------|-------------------------|
| code         | string (unique)     | Upgrade membership code |
| retail_tier  | string              | Tier mapping            |
| expires_at   | datetime            | Expiry timestamp        |
| account_id   | string (nullable)   | Activated account       |
| activated_at | datetime (nullable) | Activation timestamp    |



- Code State Definition
  - Valid:
    - `activated_at IS NULL`
    - `expires_at > NOW()`
  - Used:
    - `activated_at IS NOT NULL`
  - Expired:
    - `expires_at < NOW()`

#### 4.2.6. Cache Layer

- Key: `BZB_USER_CARD_<accountId>`
- Behavior:
  - Deleted after successful BZB upgrade
  - Ensures refreshed persona/tier data

## 5. Functional Design

### 5.1. Invitation Code

#### 5.1.1. Login with Invitation Code

**5.1.1.1. Narrative:** Enable users to access the One Bangkok Application using an Invitation Code that grants temporary access and associated privileges. The Invitation Code is reusable until expiration and affects profile editing and role assignment rules.

**5.1.1.2. User Story:** As a user, I want to log in using an invitation code so that I can access the app without registering with email or phone initially.

#### 5.1.1.3. Acceptance Criteria

- **Input Rules**
  - Invitation Code length = 10 characters
  - Character set: A–Z, 0–9 (uppercase only)
  - Input field auto-converts to uppercase
  - Input field limits maximum length to 10 characters
- **Login Behavior**
  - User has option to log in with Invitation Code
  - Entering a valid code authenticates the user
  - User must accept T&C and Privacy Policy before proceeding
  - User can navigate back to previous screen

- **Account Completion**
  - User must select Email or Phone to complete account
  - Email / Phone must follow existing application validation rules
  - Email / Phone must not already exist in system
  - Verification (OTP) is required before setting password
  - Password setup follows existing application criteria
  - User must select persona: Thai or Non-Thai
  - System creates account after persona selection
- **Code Deactivation**
  - Code is deactivated only after successful account creation
  - If user kills app or leaves before completion → code remains active
- **Tier Assignment**
  - Tier assigned based on code configuration: Influencer, Ambassador
- **Post-Registration Behavior**
  - Invited users can add Email and Phone in Settings
  - After adding identity, system updates role and privileges automatically

#### 5.1.1.4. Error Handling

- Empty input → display error message
- Invalid code → display “Invalid Invitation Code”
- Used code → display “This code has already been used”
- Expired code → display “This code has expired”



## 5.1.2. Disable Name Editing for Invited User

**5.1.2.1. User Story:** As a user who logs in via Invitation Code, I should not be able to edit my first name and last name.

### 5.1.2.2. Acceptance Criteria

- In **Edit Profile > Edit Name**
  - First Name = read-only
  - Last Name = read-only
  - Continue button disabled
- User can still edit:
  - ID Card / Passport No. (if not yet provided)
  - Address
  - Date of Birth
  - Gender

### 5.1.2.3. UXUI Design

9:41

← Name

First name \*

Tina

Middle name \*

Swift

Last name \*

Trends

If you need to update your name,  
please contact [024 835 555](tel:024835555)

Continue

### 5.1.3. Generate Invitation Codes

**5.1.3.1. User Story:** As Retail BU, I want to generate Invitation Codes for a specific VIP list so that each VIP can access the application using a personalized code.

#### 5.1.3.2. Acceptance Criteria

- Code is unique per individual
- Character set: A–Z, 0–9 (uppercase only)
- Length = 10 characters
- Code is non-transferable
- Code expires on last day of calendar year
- Code usable only once for activation
- Code configurable for Tier: Influencer, Ambassador

#### 5.1.3.3. Stored Data

- VIP First Name
- VIP Last Name
- Validity Period
- Tier
- Status (Active / Deactivated / Expired)

## 5.2. Upgrade Membership

### 5.2.1. Upgrade Membership with Code

**5.2.1.1. Narrative:** As-is the upgrade membership feature accessed by users via embedded webviews. This task allows users to upgrade their retail membership tier using a special Upgrade Code provided by Retail BU via application directly.

**5.2.1.2. User Story:** As a user, I want to upgrade my retail tier using an upgrade code so that I can upgrade my retail tier with code received .

### 5.2.1.3. Acceptance Criteria

- **Access**
  - User opens My Profile
  - User taps “Upgrade Membership”
- **Input Rules**
  - Length = 10 characters
  - Character set: A–Z, 0–9
  - Auto-uppercase
  - Required field
- **Success Behavior**
  - Display success screen
  - Display upgraded tier
  - If user presses Back → input cleared
  - Using a new code overrides previous tier
- **Error Handling**
  - Empty or invalid code → display error
  - System error / timeout → display “Something went wrong”
- **Tier Result**
  - Tier assigned based on code configuration: Influencer, Ambassador

### 5.2.1.4. UXUI Design: Full Access via Link: Figma



### 5.2.2. Generate Invitation Codes

**5.2.2.1. User Story:** As Retail BU, I want to generate Upgrade Membership Codes so that users can upgrade their tier based on code configuration..

#### 5.2.2.2. Acceptance Criteria

- Code is unique
- Character set: A–Z, 0–9 (uppercase only)
- Length = 10 characters
- Code expires on last day of calendar year
- Code usable only once for tier upgrade
- Code configurable for Tier: Influencer, Ambassador

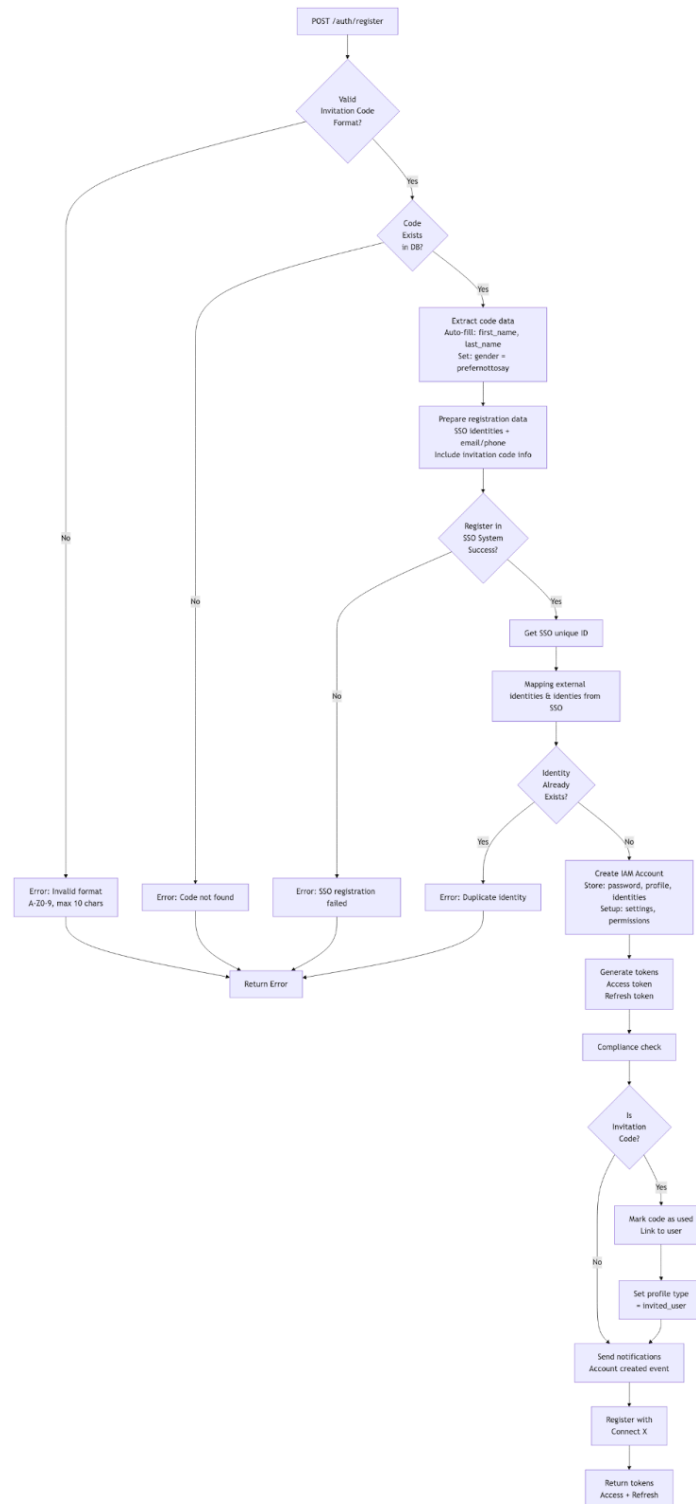
#### 5.2.2.3. Stored Data

- Validity Period
- Tier
- Status (Active / Deactivated / Expired)

## 6. Data Flow

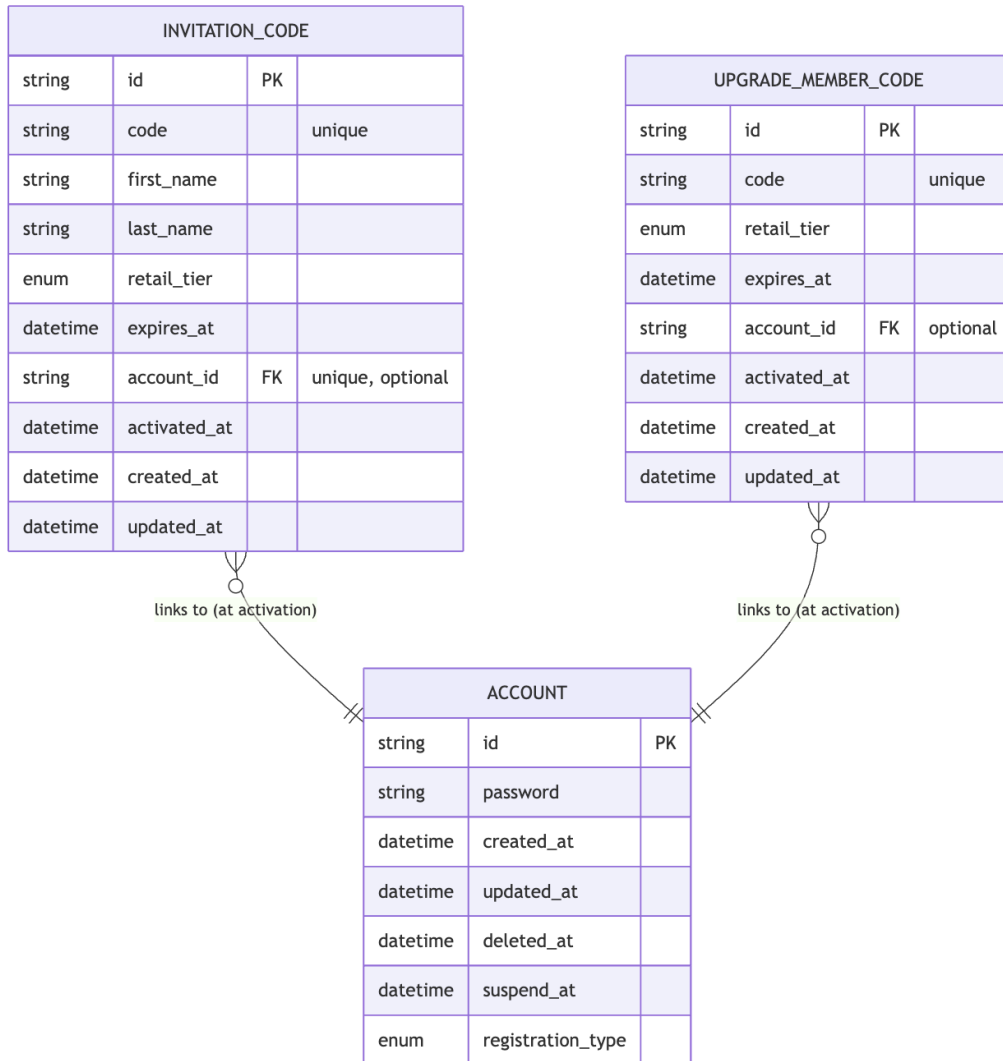
### 6.1. Invitation Code

#### 6.1.1. Registration with invitation-code Flow

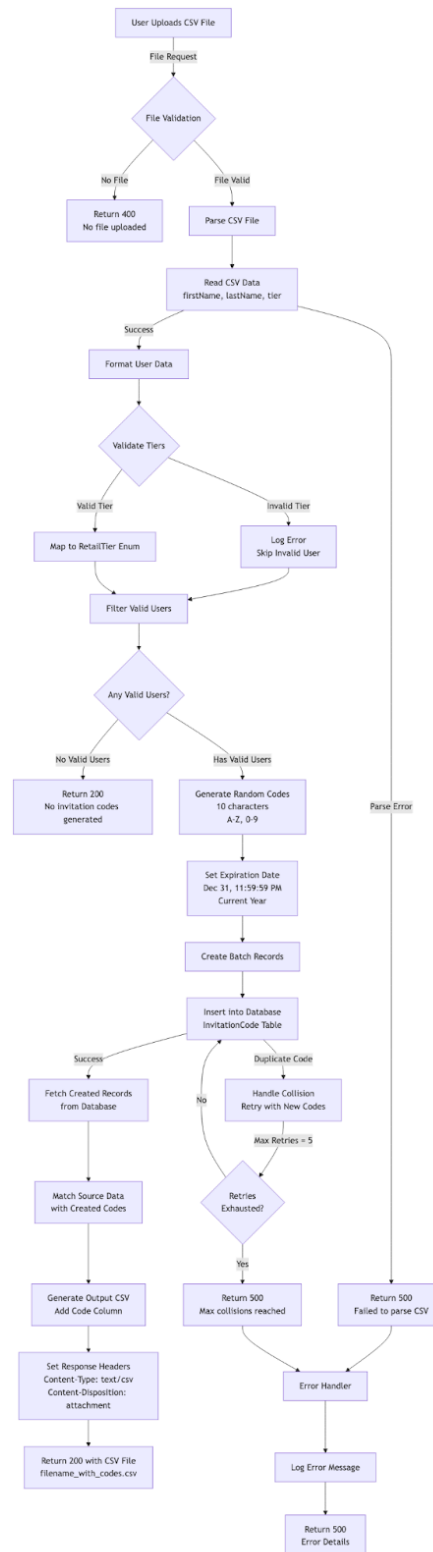




### 6.1.2. Table ER - Diagram

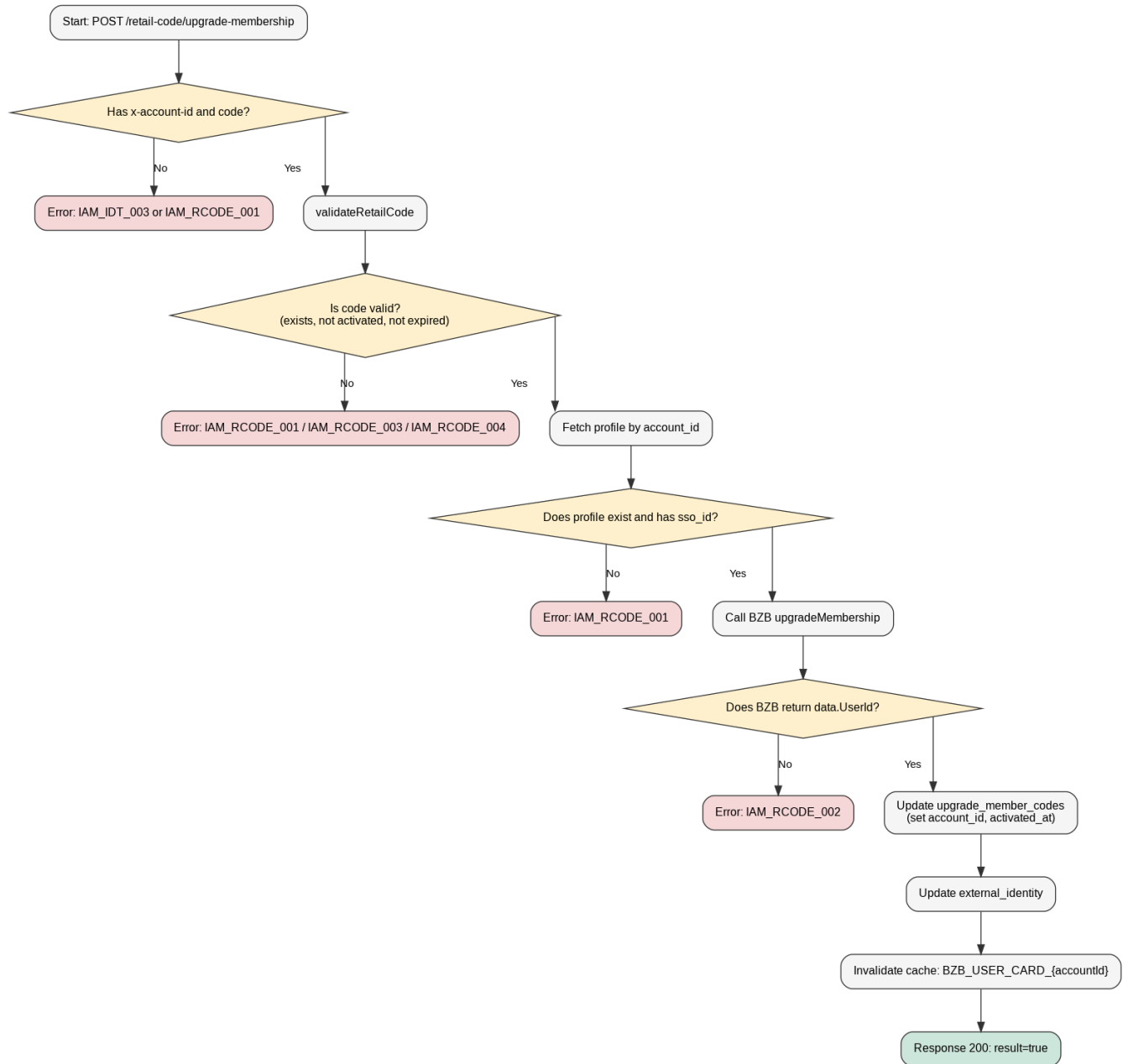


### 6.1.3. Generate Invitation Code Flow

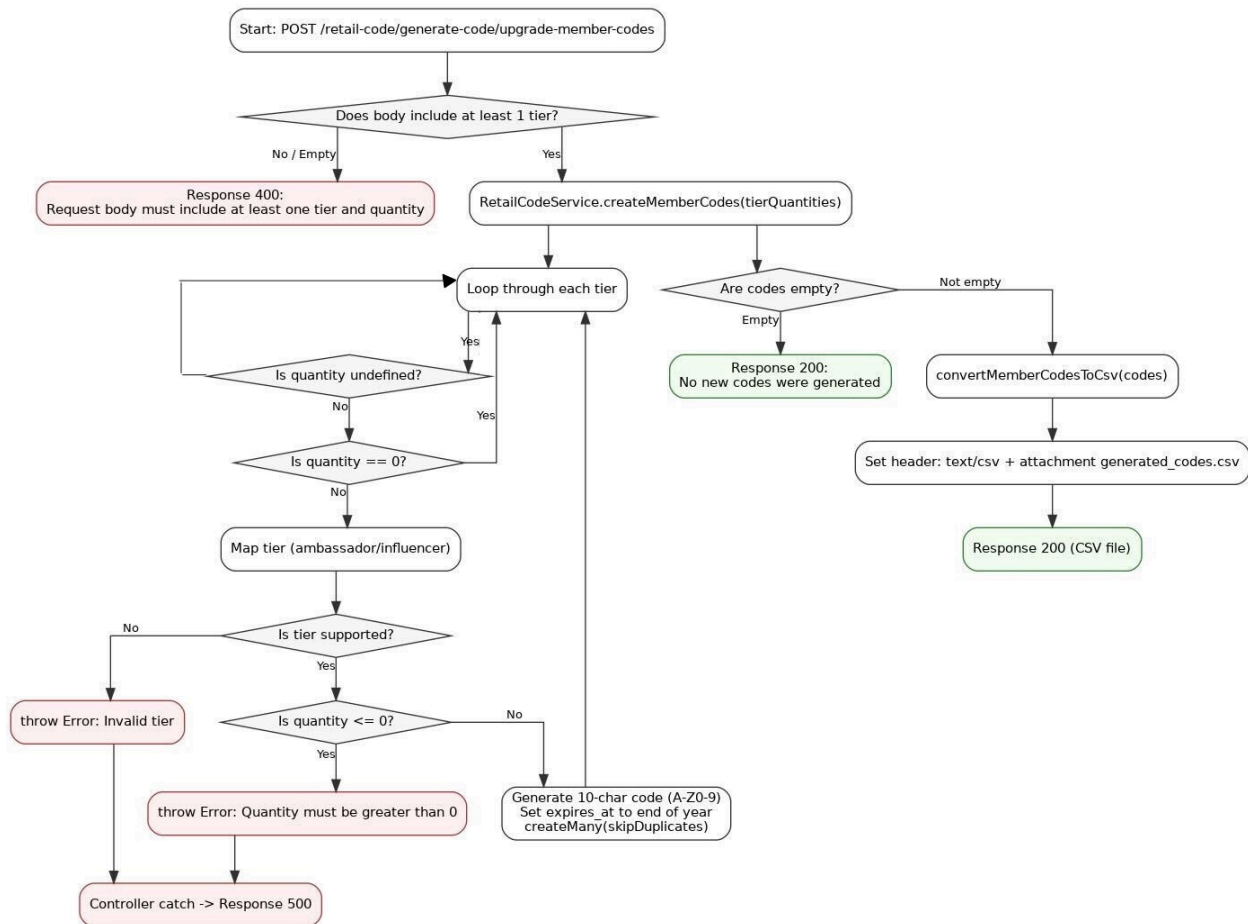


## 6.2. Upgrade Membership

### 6.2.1. Upgrade Membership Flow



### 6.2.2. Generate Code Upgrade Flow



## 7. Example Request/Response

### 7.1. Request - Invitation Code Registration

```

POST {iam_base_url}/auth/register
Content-Type: application/json
app-version: 2.8.0

{
  "registerMethod": "invitation-code",
  "invitationCode": "ABC123XYZ9",
  "profile": {
    "dob": "1990-01-15",
    "title": "mr"
  },
  "identities": {
    "provider": "email",
    "identifier": "user@example.com"
  },
  "password": "SecurePass123!",
  "device": {
    "device_id": "uuid-device-123",
    "os": "iOS"
  },
  "push_token": {
    "value": "fcm_token_abc123xyz",
    "type": "fcm"
  },
  "sso_register": {
    "emails": [
      {
        "value": "user@example.com",
        "isDefault": true
      }
    ],
    "phoneNumbers": [
      {
        "value": "+66812345678",
        "isDefault": true
      }
    ],
    "addresses": [],
    "external": []
  },
  "receive_updates": true
}

```

**7.1.1. Response (Success - 200)**

```
"data": {
  "token": {
    "value": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  },
  "refreshToken": {
    "value": "refresh_token_xyz789abc..."
  }
}
```

**7.1.2. Response (Error - 400 - Invalid Code Format)**

```
{
  "message": "Invalid request body",
  "status": 400
}
```

**7.1.3. Response (Error - 400 - Code Not Found)**

```
{
  "message": "Code not found",
  "status": 404
}
```

**7.1.4. Response (Error - 500 - Duplicate Identity)**

```
{
  "message": "Identity already exists",
  "status": 500
}
```

**7.1.5. Response (Error - 400 - SSO Registration Failed)**

```
{
  "message": "External SSO system registration failed",
  "status": 400
}
```

### 7.1.6. Error Scenarios

| Error                          | Status | Message                    | Cause                                                   |
|--------------------------------|--------|----------------------------|---------------------------------------------------------|
| Invalid invitation code format | 400    | "Invalid request body"     | Code contains lowercase, special chars, or wrong length |
| Invitation code not found      | 400    | "Code not found"           | Code doesn't exist in database                          |
| Code already activated         | 400    | "Code already activated"   | The invitation code already used                        |
| Code expired                   | 400    | "Code expired"             | This invitation code already expired                    |
| Duplicate email/phone          | 500    | "Identity already exists"  | Email or phone already registered                       |
| Missing password               | 500    | "Password cannot be null"  | Password required but not provided                      |
| Invalid tier in code           | 400    | "Invalid retail tier"      | Code has invalid tier value                             |
| Database error                 | 500    | "Failed to create account" | Database operation failed                               |

### 7.1.7. HTTP Status Codes

| Code | Meaning      | When Used                                    |
|------|--------------|----------------------------------------------|
| 200  | Success      | Account created, tokens returned             |
| 400  | Bad Request  | Invalid code format, missing required fields |
| 404  | Not Found    | Invitation code not found                    |
| 500  | Server Error | SSO error, database error, system failure    |

## 7.2. Request - Generate Invitation code

```

POST {iam_base_url}/generate-code/invitation-codes
Content-Type: multipart/form-data

--boundary

Content-Disposition: form-data; name="file"; filename="users.csv"
Content-Type: text/csv
firstName,lastName,tier
John,Doe,influencer
Jane,Smith,influencer
Bob,Johnson,ambassador
--boundary-
  
```

### 7.2.1. Response (Success - 200)

```

FirstName, LastName, Tier, Code
John, Doe, influencer, A3X9K2L7M1
Jane, Smith, influencer, B5Y8N4P6Q9
Bob, Johnson, ambassador, C7W1J3R8T2
  
```

### 7.2.2. Response (Error - 500)

```

{
  "message": "Failed to generate invitation codes: Failed to create invitation
codes due to repeated collisions."
}
  
```

### 7.2.3. Error Scenarios

| Error            | Status | Message                                                                                              | Cause                        |
|------------------|--------|------------------------------------------------------------------------------------------------------|------------------------------|
| No file uploaded | 400    | "No file uploaded"                                                                                   | Missing file in request      |
| CSV parse error  | 500    | "Failed to generate invitation codes: {error}"                                                       | Malformed CSV                |
| No valid users   | 200    | "No new invitation codes were generated"                                                             | All users have invalid tiers |
| Max collisions   | 500    | "Failed to generate invitation codes: Failed to create invitation codes due to repeated collisions." | Code generation conflicts    |
| Database error   | 500    | "Failed to generate invitation codes: {error}"                                                       | Database operation failed    |

## 7.3. Request - Upgrade Membership

```

POST {iam_base_url}/retail-code/upgrade-membership
Content-Type: application/json
x-access-token: Bearer <JWT>

{
  "code": "TESTCODEJA"
}
  
```

### 7.3.1. Response (Success - 200)

```

{
  "data": {
    "result": true,
    "message": "Upgrade member code activated
successfully"
  }
}
  
```



### 7.3.2. Error Scenarios

| Error Code    | Message                                       | Remark                       |
|---------------|-----------------------------------------------|------------------------------|
| IAM_RCODE_001 | Invalid Code, Cannot found profile, No SSO ID | Display Error Message        |
| IAM_RCODE_003 | Code taken                                    | Display Error Message        |
| IAM_RCODE_004 | Code Expried                                  | Display Error Message        |
| IAM_RCODE_002 | Call BZB Fail                                 | Display Something went wrong |

### 7.4. Request - Generate Upgrade Membership code

```
POST {iam_base_url}retail-code/generate-code/upgrade-member-codes

{
  "ambassador": 100,
  "influencer": 50
}
```

#### 7.4.1. Response (Success)

```
code : Tier,Code
      ambassador,TESTCODE12
      influencer,TESTCODE34
```

- Return file .csv (`Content-Type: text/csv`)

#### 7.4.2. Response (Error)

| HTTP Status | Cause                                       | Message                                                                 | Relate Code                                                                                                                                 |
|-------------|---------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 400         | Not sent request body or body empty         | Request body must include at least one tier and quantity                | src/controllers/retail_code_controller.ts: 86                                                                                               |
| 500         | Tier not support or fail quantity condition | - Invalid tier: insider<br>- Quantity for <tier> must be greater than 0 | - throw:<br>src/services/retail_code_service/index.ts (createMemberCodes), catch and 500:<br>src/controllers/retail_code_controller.ts: 104 |